

area; hence, they are allocated in this segment. So, the lifetime of a local static variable is that of the lifetime of the program!

There are two parts in this data segment itself: the initialised data segment and the uninitialised data segment.

When the variables are initialised to some value (other than the default value, which is zero), they are allocated in the initialised data segment.

When the variables are uninitialised, they get allocated in the uninitialised data segment. This segment is usually referred to with a cryptic acronym called BSS. It stands for Block Starting with Symbol, and gets its name from the old IBM systems which had that segment initialised to zero.

The data area is separated into two, based on explicit initialisation, because the variables that are to be initialised need to be initialised one by one. However, the variables that are not initialised need not be explicitly initialised with zeros, one by one. Instead, the job of initialising the variables to zero is left to the operating system to take care of. This bulk initialisation can greatly reduce the time required to load.

When we want to run an executable program, the OS starts a program known as a loader. When this loads the file into memory, it takes the BSS segment and initialises the whole thing to zeros. That is why (and how) the uninitialised global data and static data always get the default value of zero.

The layout of the data segment is in the control of the underlying operating system; however, some loaders give partial control to the users. This information may be useful in applications such as embedded systems.

The data area can be addressed and accessed using pointers from the code. Automatic variables have an overhead in initialising the variables each time they are required, and code is required to do that initialisation. However, variables in the data area do not have such runtime overhead because the initialisation is done only once and, that too, at loading time.

Code segment

The program code is where the executable code is available for execution. This area is also known as the 'text segment' and is of fixed size. This can be accessed only by function pointers and not by other data pointers. Another important piece of information to take note of here is that the system may consider this area as a 'read only' memory area, and any attempt to write in this area can lead to undefined behaviour.

Stack and heap segments

To execute the program, two major parts are used: the stack and heap. Stack frames are created in the stack for functions and in the heap for dynamic memory allocation. The stack and heap are uninitialised areas. Therefore, whatever happens to be in the memory becomes the initial (garbage) value for the objects created in that space.

The local variables and function arguments are allocated in the stack. For the local variables that have an initialisation value, code is generated by the compiler to initialise them explicitly to those values when the stack frames are created. For function parameters, the compiler generates code to copy

the actual arguments to the space allocated for the parameters in the stack frames. (**Note:** Compilers do this efficiently, so this description is not entirely correct; we have given this description because it is useful to understand how function parameters are treated as local variables).

An example

We'll take a sample source program and see where different program elements are stored when that program executes. The comments in the program explain where the variables get stored.

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

int bss1;
static double bss2;
char *bss3;
// these variables are stored in initialized to zero segment
// also known as uninitialized data segment (BSS)

int init1 = 10;
float init2 = 10.0f;
char *init3 = "hello world";
// these variables are stored in initialized data segment
// the code for main function gets stored in code segment
int main() {
    int local1 = 10;
    // this variable is allocated in stack; initialization code is generated by the
    // compiler
    int local2;
    // this variable is not initialized; hence it has garbage value
    // remember: it does not get initialized to zero

    static int local3;
    // this is allocated in BSS segment and gets initialized to zero

    static int local4 = 100;
    // this gets allocated in initialized data segment

    int ("local_foo")(const char *, ...) = printf;
    // printf is in a shared library (libc or C runtime library)
    // local_foo is a local variable (a function pointer) that
    // points to that printf function

    local_foo("hello world\n");
    // this function call results in creation of a 'stack frame'
    // in the stack area

    int *local5 = malloc(sizeof(int));
    // local5 is allocated in stack; however it points to a dynamically
    // allocated block in heap

    return 0;
    // the stack frame for main function gets destroyed after executing main
}
```